MFE FORMWORK MR11 SYSTEM

Code Guide: Frontend

Version 1.0

Prepared for

MFE Formwork

Prepared by

MyIntellX project team

Date

21 October 2026

Status

Draft for client review

Document control

Item	Detail
Document	Code Guide: Frontend
System	MFE Formwork MR11 System, version 1.0
Document version	1.0
Date	21 October 2026
Status	Draft for client review
Based on	Repository izzatmyintellx05/MFE, branch main, commit 0cc8c36
Prepared for	MFE Formwork
Prepared by	MyIntellX project team
Intended readers	Developers taking over the code, client IT
Classification	Confidential, for client and vendor use only

Revision history

Version	Date	Description	Author
1.0	21 October 2026	First issue for client review	MyIntellX project team

Contents

1. Introduction 4
2. Before you start 4
3. How the frontend fits together 5
3.1 What happens when a page opens 5
3.2 Where things live 5
4. The application shell 6
4.1 src/main.tsx 6
4.2 Routes and access checks (src/App.tsx) 6
4.3 Who can open what (src/utils/access.ts) 6
4.4 The sign-in state (src/store/auth.store.ts) 7
4.5 The API client (src/api/client.ts) 7
4.6 src/types/index.ts 7
5. Layout 7
5.1 The sidebar (components/layout/Sidebar.tsx) 7
5.2 components/layout/AppLayout.tsx, which is no longer used 8

6. Pages 8
6.1 Signing in (pages/Login.tsx) 8
6.2 The MR11 Master Schedule (pages/Mr11Dashboard.tsx) 8
6.3 The CEO Executive Hub (pages/CeoDashboard.tsx) 9
6.4 Department workbooks (pages/DepartmentPage.tsx) 9
6.5 Planning and Production (pages/PlanningPage.tsx and pages/ProductionPage.tsx) 10
6.6 Admin Permissions (pages/AdminPage.tsx) 10
6.7 Pages that are not used 10
7. Workbook components 10
7.1 The MR11 table (components/workbook/Mr11Table.tsx) 10
7.2 The side panel (components/workbook/Mr11SidePanel.tsx) 11
7.3 Column groups (components/workbook/ColumnGroupsMenu.tsx) 11
7.4 Department highlighting (components/workbook/DepartmentHighlight.tsx) 11
7.5 Deleted rows (components/workbook/DeletedRowsMenu.tsx) 11
7.6 The workbook viewer (components/workbook/HighFidelityViewer.tsx) 11
7.7 Approval history (components/workbook/ApprovalHistoryTable.tsx) 12
7.8 Monthly dispatches (components/workbook/DispatchMonthlyTable.tsx) 12
8. Common components 12
9. Utilities 12
9.1 utils/formatNumber.ts 12
9.2 utils/designDates.ts 13
9.3 utils/mr11Filters.ts 13
9.4 utils/ceoMetrics.ts 13
9.5 utils/usePersistentState.ts 14
10. Styles and static files 14
11. Making common frontend changes 14

1. Introduction

This guide explains every file in the React frontend under `src/`. For each page and component it describes what it does, where its data comes from and which helper functions sit behind it. It assumes you are comfortable with React hooks and TypeScript.

We suggest reading section 3 for the big picture first, and then turning to the file you need. The code excerpts are copied from commit `0cc8c36`. The server side is covered in Code Guide: Backend, and the project settings, build and scripts in Code Guide: Configuration, Build and Scripts.

2. Before you start

Item	Detail
Framework	React 19 with React Router 7
Build tool	Vite 8, set up in `vite.config.ts`. During development the Express server serves the app through Vite on port 3000
Styling	Tailwind CSS 4 utility classes written in the JSX, plus `src/index.css`
State	Component state with hooks, the sign-in state in a Zustand store, and page settings in localStorage
Charts	Chart.js through react-chartjs-2, used only by the CEO Executive Hub
Icons	lucide-react
API calls	A single Axios instance, `src/api/client.ts`, with the base address `/api`

3. How the frontend fits together

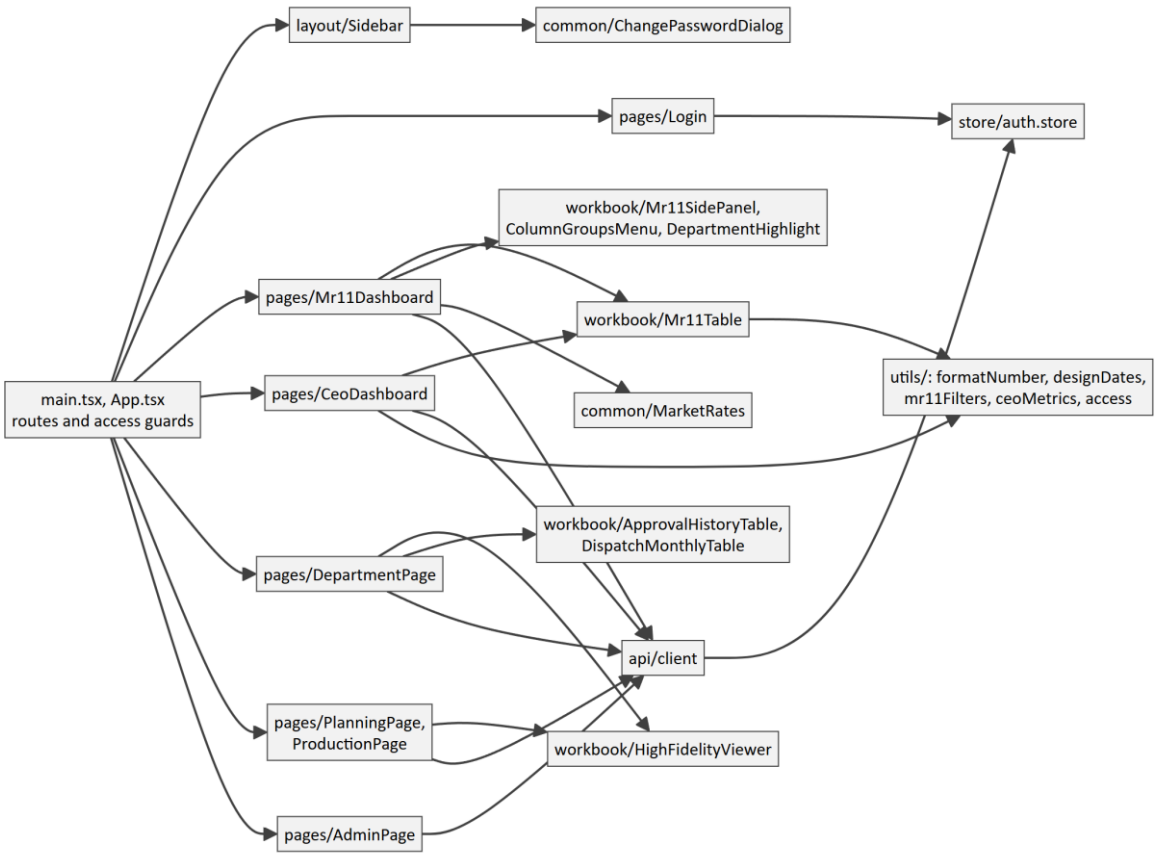


Figure 1: The frontend code map, showing the pages, the components they use, and the shared state and utilities

3.1 What happens when a page opens

1. `src/main.tsx` draws `<App />` into the `#root` element of `index.html`.
2. `App.tsx` reads the stored token. If there is one, `initAuth()` calls `GET /api/auth/me` to refresh the user and their roles.
3. The router chooses the page. `ProtectedLayout` sends anyone without a token to `/login`, and `RequireAccess` sends anyone without the right role to `/mr11`.
4. The page loads its data through `api`, which adds the `Authorization: Bearer <token>` header. It keeps the data in `useState` and draws its components.
5. If any call other than sign-in gets a 401 answer, the user is signed out and taken back to `/login`.

3.2 Where things live

Folder	What it holds
`src/pages/`	One file for each screen
`src/components/layout/`	The sidebar, and a layout that is no longer used
`src/components/workbook/`	The MR11 table and its side panel, the workbook viewer and the history tables

Folder	What it holds
src/components/common/	The market cards, the password dialog, the logo, and the zoom and full-screen buttons
src/store/	The sign-in store
src/api/	The Axios client
src/utils/	Access rules, number formats, filters, CEO figures, design-date colours and remembered settings
src/types/	Shared TypeScript types

4. The application shell

4.1 src/main.tsx

This file mounts the app inside React's StrictMode and loads index.css.

4.2 Routes and access checks (src/App.tsx)

Part	What it does
ProtectedLayout	Sends the user to /login when there is no token. It also checks localStorage, so the sign-in page doesn't flash up when the page is refreshed. Otherwise it draws the Sidebar next to the page
RequireAccess	Waits for the user to load, then shows the page if allow(user) is true, and otherwise sends the user to /mr11
DepartmentRoute	Reads :code from the address and checks canSeeDepartment(user, code)
The routes	/login, /mr11, /admin (for isAdmin), /ceo (for canSeeCeoDashboard), /departments/PLANNING, /departments/PRODUCTION and /departments/:code. Any other address goes to /mr11

```
<Route path="/ceo" element={
  <ProtectedLayout>
    <RequireAccess allow={canSeeCeoDashboard}><CeoDashboard /></RequireAccess>
  </ProtectedLayout>
} />
```

The Planning and Production routes are declared before /departments/:code, which is how those two departments get pages of their own.

4.3 Who can open what (src/utils/access.ts)

Function	The rule it applies
rolesOf(user)	Returns the user's role codes in capital letters
isAdmin(user)	True if the user has ADMIN or is admin@mfeformwork.com
canSeeCeoDashboard(user)	True for ADMIN or CEO
canSeeDepartment(user, code)	True for ADMIN or the department's own role. Shell Plan and Design accept either of the two roles, as listed in DEPARTMENT_ROLES

These rules only decide what the screens show. The server applies the same rules to the data itself.

4.4 The sign-in state (src/store/auth.store.ts)

This is a Zustand store that every component shares.

Member	What it does
`user`, `token` and `usingDefaultPassword`	Start from the values in localStorage (`mfe_user`, `mfe_token` and `mfe_default_password`), so the user stays signed in after a refresh
`login(email, password)`	Calls `POST /auth/login`, stores the token and the user, and remembers whether the default password is still in use
`changePassword(current, next)`	Calls `POST /auth/change-password` and clears the default-password flag
`logout()`	Clears the stored values and the header, then loads `/login`
`initAuth()`	Calls `GET /auth/me` to refresh the user. If that fails, it quietly clears the session

4.5 The API client (src/api/client.ts)

This is an Axios instance with `baseURL: '/api'`. A request interceptor adds the token from the store. A response interceptor signs the user out when it sees a 401, except on the sign-in call itself, so that the sign-in page can show "Wrong password" instead.

```
api.interceptors.response.use((response) => response, (error) => {
  const isLoginRequest = String(error.config?.url || '').includes('/auth/login');
  if (error.response?.status === 401 && !isLoginRequest) useAuthStore.getState().logout();
  return Promise.reject(error);
});
```

4.6 src/types/index.ts

This file holds the shared types `RoleCode`, `FileVersionStatus`, `Department` and `DepartmentVersion`. Most pages type their API data as `any`, so these types serve more as documentation than as a check.

5. Layout

5.1 The sidebar (components/layout/Sidebar.tsx)

The sidebar is the navigation on the left of every screen, and it only shows the links the user's roles allow.

Part	What it does
`DEPARTMENT_LINKS`	The six department links, each with the roles that may see it
`SideLink`	A single link. It shows an icon and a label when the sidebar is open, and an icon with a tooltip when it is collapsed. The current page is highlighted
Collapsing	Whether the sidebar is open is remembered as `sidebar.open`
Filtering by role	The CEO Executive Hub link appears if `canSeeCeoDashboard` allows it, Admin Permissions if `isAdmin` does, and only the departments that `canSeeDepartment` allows
The footer	The user's name and email, Change password (shown in amber while the default password is still in use) and Sign out

Part	What it does
ChangePasswordDialog	Opens from the key icon

If the user has a token but their roles haven't loaded yet, the sidebar calls `initAuth()`.

5.2 components/layout/AppLayout.tsx, which is no longer used

An `<Outlet />` layout from an earlier design. `App.tsx` uses `ProtectedLayout` instead.

6. Pages

6.1 Signing in (pages/Login.tsx)

The page has a form for the corporate email address and password. `performLogin()` checks that both are filled in, calls `login()` from the store, and opens `/mr11` when it succeeds. Errors show the server's message. On `WRONG_PASSWORD` the password box is cleared and the cursor is put back in it. A network error explains that the server can't be reached.

6.2 The MR11 Master Schedule (pages/Mr11Dashboard.tsx)

This page shows the MR11 with the market cards, search, filters, column groups, zoom, full screen, export and Sync, and lets administrators delete and restore rows.

State	What it holds
`records`, `headers`, `headerGroups` and `numberFormats`	The data from `GET /mr11`
`fxRate`, `currencyRates` and `lmePrice`	The figures for the market cards
`columnDepartments` and `departmentColors`	What the department highlight needs
`searchTerm` and `filters`	Kept for as long as the page is open
`zoom`, `panelOpen`, `hiddenGroups` and `showRates`	Remembered in localStorage, under keys starting `mr11.`
`deletedRows`, `rowBusy` and `confirmRow`	Used when administrators delete and restore rows

Function	What it does
`fetchMr11Data()`	Loads everything from `GET /mr11`
`handleRegenerate()`	Calls `POST /mr11/regenerate` and then reloads. This is the Sync button
`handleExport()`	Downloads `GET /mr11/export` as a blob and saves it under the server's file name. A plain link wouldn't work, because the request needs the token
`handleDeleteRow(row)` and `handleRestoreRow(row)`	Call `POST /mr11/rows/delete` or `POST /mr11/rows/restore` with the row's `_rowKey`, then reload
`visibleHeaders`	The headers, leaving out any hidden column groups
`shownRecords`	Searches the project number, name and short name, then applies `applyFilters()` and then `remergeStreamBlocks()`

```
const shownRecords = useMemo(() => {
  const term = searchTerm.trim().toLowerCase();
  const searched = term ? records.filter((r) => [...].some((v) => v.includes(term))) :
records;
  return remergeStreamBlocks(records, applyFilters(searched, filters));
}, [records, searchTerm, filters]);
```

The delete button asks for confirmation first, using `confirmRow`. The icon bar on the right (`RailButton`) shows or hides the options panel and the market cards.

6.3 The CEO Executive Hub (pages/CeoDashboard.tsx)

This page gives management a view of the latest MR11. Everything on it is worked out in the browser from `GET /mr11`.

Section	What it shows	What builds it
The header and filters	The region (All, Malaysia, India or Rest of World) and the month	`filterByRegion()` and `monthColumns()`
Monthly m² · Actual vs Forecast	A line chart with a "Today" divider, and hollow points wherever the 100,000 m² default is used	`monthlySeries()`, and the Chart.js plugin `todayDivider`
Pricing & Market	The average selling price per m² in USD and MYR, the USD to MYR rate, and the LME price	`averageSellingPrice()` and the live rates
Production Pipeline	Total Design, Processed, Produced, Dispatched and Sailed, or the pipeline for a single month	`stageTotals()` and `monthPipeline()`
Breakdown by region	The stage totals for each region	`stageTotals()`, run once for each region
Raw data	The MR11 rows for the chosen region	`Mr11Table`

The chart styling (colours, fonts, faded fills and short axis labels such as 100k) is set at the top of the file.

6.4 Department workbooks (pages/DepartmentPage.tsx)

This page shows and uploads the workbooks for BD, Finance, Shell Plan & Design and Dispatch.

Part	What it does
The department code	Comes from the address. SHELLPLAN is shown as the combined DESIGN workbook
`fetchDepartmentData()`	Calls `GET /departments/:code` and shows the sheets of the active version
`handleFileUpload()`	Sends the file as form data to `POST /departments/:code/upload`, adding `?part=local` or `?part=overseas` for Dispatch, and then reloads
Dispatch	A Local and Overseas switch (`dispatch.part`), with `dispatchFileOf()` choosing that file's sheets. The Monthly dispatched switch (`dispatch.showMonthly`) shows `DispatchMonthlyTable`
Shell Plan & Design	The approval history switch (`design.showApprovalHistory`, on by default) shows `ApprovalHistoryTable`
The viewer	`HighFidelityViewer`, with zoom and full screen

6.5 Planning and Production (pages/PlanningPage.tsx and pages/ProductionPage.tsx)

Both pages load their department (`GET /departments/PLANNING` or `GET /departments/PRODUCTION`) and its series history (`GET /mr11/planning-series` or `GET /mr11/production-series`). Each has two tabs. **Active Sheet** shows the workbook in `HighFidelityViewer`, and **Series Ledger** (on Planning) or **Production Series** (on Production) shows a table of the series with their signature colour, quantities and dates. The upload button is labelled **Upload & Sync**, and it tells the user if the file couldn't be saved. Each page remembers its own zoom level, as `zoom.PLANNING` or `zoom.PRODUCTION`.

6.6 Admin Permissions (pages/AdminPage.tsx)

Function	What it calls	What it does
`fetchUsers()`	GET /admin/users	Loads the table of users and their permissions
Provision New User	POST /admin/users	Takes a name, email, password and a preset from `DEPARTMENT_PRESETS`
Changing the preset	PATCH /admin/users/:id/permissions, with `primaryProfile`	Replaces the user's roles with the preset's roles
A department switch	PATCH /admin/users/:id/permissions, with `roles`	Adds or removes one department role
Delete	DELETE /admin/users/:id	Runs after a confirmation box. The main administrator is shown as protected

`detectActivePreset(roles)` shows which preset a user's roles match, or Custom Permissions if they match none. The CEO role doesn't appear in either the preset list or the department list.

6.7 Pages that are not used

`Register.tsx` and `PendingAccess.tsx` (for self-registration) and `VisualizationDashboard.tsx` (an early dashboard built on `/api/visualization`) have no routes, so nobody can reach them.

7. Workbook components

7.1 The MR11 table (components/workbook/Mr11Table.tsx)

This is the MR11 grid. The MR11 page uses it, and so does the raw data section of the CEO Executive Hub.

Behaviour	How it is done
A two-row header	Columns in a `HeaderGroup` share a top cell, and the other columns span both rows
Month columns	Show ACTUAL or F'CAST under the header, using `monthColumnKind()`
Numbers	`formatCellNumber()` adds comma separators and uses the fixed decimal places in `numberFormats`. Identifier columns are left as they are, and payment amounts get a dollar sign
Row colours	The BD row's font colour and fill colour are used across the whole row
Merged cells	The Shell Plan and Design columns use `rowSpan={_streamSpan}`, and rows with a span of 0 leave those cells out
Design dates	`DesignDates` colours each level's date with `designDateColor()`

Behaviour	How it is done
Highlighting	`headerGlow()` and `columnGlow()` outline the columns of the chosen departments
Values on several lines	ETD/ATD keeps its line breaks
The delete button	Shown when `onDeleteRow` is passed in, which only happens for administrators
Zoom	Uses the CSS `zoom` property on the element around the table

7.2 The side panel (components/workbook/Mr11SidePanel.tsx)

The panel on the right has three sections that fold away, and each remembers whether it is open. **View** lists the column groups through `ColumnGroupsList`, **Highlight columns** shows `DepartmentLegend`, and **Filters** holds the filters. Each filter (`FilterGroup`) lists the values found in the rows along with their counts, has a search box when it has more than 8 values, and lets the user tick values on and off. **Clear all filters** empties all of them.

7.3 Column groups (components/workbook/ColumnGroupsMenu.tsx)

Export	What it does
`columnGroupOf(header, columnDepartments)`	Gives the group a column is shown or hidden with. BD's columns are split into project details, payment terms, pricing & LME, and monthly actual and forecast. Other columns are grouped by department. The columns that identify the project belong to no group, so they are always shown
`columnGroups(departmentColors)`	The list of groups, with their labels and colours
`ColumnGroupsList`	The list of tick boxes

7.4 Department highlighting (components/workbook/DepartmentHighlight.tsx)

`useDepartmentHighlight()` decides which columns are outlined. A department user starts with their own department switched on, and Shell Plan or Design switches on both of those. ADMIN and CEO start with none. `toggle()` switches a department on or off, and `highlightColumns` maps each column to its colour. `DepartmentLegend` draws the colour key, which the user can click.

```
if (roles.includes('ADMIN') || roles.includes('CEO')) return [];
const own = roles.filter((r) => r in DEPARTMENT_LABELS);
if (own.includes('SHELLPLAN') || own.includes('DESIGN')) return [...new Set([...own,
'SHELLPLAN', 'DESIGN'])];
return own;
```

7.5 Deleted rows (components/workbook/DeletedRowsMenu.tsx)

This is a menu for administrators that lists the rows removed from the MR11, showing the project, stream and product type and who removed each one and when. Each row has a **Restore** button.

7.6 The workbook viewer (components/workbook/HighFidelityViewer.tsx)

The viewer draws a parsed sheet as an HTML table that looks like Excel. It works through six steps:

1. It builds a map of the cells, keyed by "row_col", and a set of the cells that are covered by merges.

2. It finds the last row and column that hold a value, so that blank formatted rows at the bottom of a template aren't drawn. As a safety net, it stops at 5,000 rows and 200 columns.
3. It draws the column letters with `columnLetter()` (A to Z, then AA and onwards) and the row numbers.
4. It draws each cell with its fill, font colour, bold, italics, alignment, wrapping and merge span. Numbers get comma separators unless the column holds identifiers.
5. `headerDate()` shows day headers that were saved as Excel date numbers (46357, for example) as dates such as "1 Dec 2026".
6. The tabs at the bottom switch between sheets.

7.7 Approval history (components/workbook/ApprovalHistoryTable.tsx)

This loads `GET /mr11/shellplan-history` and lists every Approved and Reapprove entry, with the project, project number, stream, status, revision, approved and submitted figures, and the date each was first seen. A project picker narrows the list to one project, and `reloadKey` refreshes it after an upload.

7.8 Monthly dispatches (components/workbook/DispatchMonthlyTable.tsx)

This loads `GET /mr11/dispatch-monthly` and shows the m² dispatched in each month for every MR11 row. It has a filter for Local, Overseas or both, shows a total for each month, and marks the current month's figure as "so far". `monthLabel()` turns "2026-10" into "Oct-26".

8. Common components

File	What it does
`common/MarketRates.tsx`	`ExchangeRateCard` shows 1 USD in MYR and in other currencies, with BD's currencies marked. `LmePriceCard` shows the cash price, the 3-month price and the change since the previous day. Each card has a `LiveBadge` and shows its source and date
`common/ChangePasswordDialog.tsx`	Asks for the current, new and confirmed passwords, each of which can be shown or hidden. It checks the length, that the two new entries match and that the new password is different, before calling the store's `changePassword()`. It shows the server's error message if there is one
`common/BrandLogo.tsx`	The Doka MFE logo image, with a drawn version to fall back on if the image is missing
`common/ZoomControls.tsx`	Zoom out, zoom in and reset buttons, kept within a minimum and a maximum
`common/FullscreenButton.tsx`	Puts a given element into full screen with the browser's Fullscreen API. Pressing Esc returns to normal

9. Utilities

9.1 utils/formatNumber.ts

`formatNumber(n, places)` adds comma separators. Whole numbers show no decimal places and other numbers show three, unless `places` says otherwise. `formatCellNumber(value, header, places)` does

the same for a table cell, but leaves identifier columns alone. `ID_COLUMN` lists these, and they include project no, PO, NCA, stream, series, revision, code and year.

9.2 utils/designDates.ts

`todayInMalaysia()` gives today's date in Asia/Kuala_Lumpur. `monthColumnKind(header)` returns ACTUAL for a month column whose month has ended, and F'CAST for any other. `designDateColor(part)` chooses dark green when a level is complete and its date has passed, yellow when it is complete but its date is still ahead, and red when it isn't complete.

9.3 utils/mr11Filters.ts

Export	What it does
`FILTER_FIELDS`	The 13 filters. The Dispatch stage filter is worked out from the ETD/ATD text by `dispatchStage()`
`filterValue(field, row)`	The row's value for a filter, or "(blank)" if it has none
`valueCounts(field, rows)`	Each value with the number of rows that have it, in sorted order
`applyFilters(rows, selected)`	Keeps a row only if, for every filter that has ticks, the row's value is one of the ticked ones
`activeFilterCount(selected)`	How many filters have ticks
`remergeStreamBlocks(all, shown)`	Works out the Shell Plan and Design merge spans again over the rows still on screen, so that hidden rows don't break the merged cells

9.4 utils/ceoMetrics.ts

Export	What it does
`regionOf(row)`	Malaysia, India or Rest of World, taken from Countries, or from the start of the project name if Countries is empty
`firstRowPerProject(rows)`	One row for each project, used for counting projects
`monthColumns(headers)` and `isMonthCompleted()`	The month columns, and whether each month has ended
`monthKeyOfDate(value)`	The month column that a date falls in
`stageTotals(rows, month)`	The Design, Processed, Produced, Dispatched and Sailed m². When a month is given, it only counts quantities whose stage date falls in that month. Design is counted once for each merged block
`monthValue()` and `monthlySeries()`	The monthly totals for the chart, using the 100,000 m² default for forecast months that are empty
`monthPipeline(rows, month)`	One month's BD forecast, Design, Processed, Produced and Dispatched figures, taken from `_monthStages`. Re-Design Only rows are kept apart, and for a month that has ended it also shows what wasn't dispatched

9.5 utils/usePersistentState.ts

This works like `useState`, except that the value is saved to localStorage under a key and read back on the next visit. If the browser blocks storage, it simply uses the default value.

```
const [value, setValue] = useState<T>(() => {
  try { const stored = localStorage.getItem(key); return stored !== null ? JSON.parse(stored)
: initial; }
  catch { return initial; }
});
useEffect(() => { try { localStorage.setItem(key, JSON.stringify(value)); } catch {} }, [key,
value]);
```

10. Styles and static files

File	What it is
`index.html`	The page shell, with `#root` and the favicon
`src/index.css`	The Tailwind import, the fonts and a few shared classes, such as the black buttons
`public/`	The logo and icons, served exactly as they are
`src/assets/`	Images left over from the Vite starter template (`react.svg`, `vite.svg` and `hero.png`), which none of the screens use

11. Making common frontend changes

Change	Where to make it
Adding a page	Create it in `src/pages/`, add a route in `App.tsx` with the right `RequireAccess`, and add a link in `Sidebar.tsx`
Changing who can see a page	`src/utils/access.ts`, together with the matching middleware on the server
Adding an MR11 filter	Add an entry to `FILTER_FIELDS` in `utils/mr11Filters.ts`
Adding a column group	`columnGroupOf()` and `columnGroups()` in `ColumnGroupsMenu.tsx`
Changing how numbers are shown	`utils/formatNumber.ts`, or `MR11_NUMBER_FORMATS` on the server for fixed decimal places
Changing the CEO figures	`utils/ceoMetrics.ts`, because the page itself only draws them

After making a change, run `npm run lint` (which checks the types) and `npm run build`, and then test the screen in the browser.